PROJECT REPORT
Software Engineering (CSE327)

Project Title: AI-Inspired E-Commerce Platform
(Projukti Mart—AI-Inspired Electronics Web Marketplace)

Group Members:

1. **Tamanna Binty Islam Shikrity - 2211378042**
2. **Suraiya Islam Ela - 2232563642**
3. **Nibedita Das - 2411860042**

Section: 12
Group Number: 02
Date: 9th September 2026
Faculty: Dr. Mohammad Rezwanul Huq (Mrh1)
Project Video Link: h
ttps://drive.google.com/file/d/1zmNsjIZ6VD5gegbUXZbjb_nNhFKw-QV9/view?usp=sharing

Table of Contents

1. Introduction

1.1 || What We Built

This Software Requirements Specification (SRS) document provides a complete description of the functional and non-functional requirements for **Projukti Mart**, an AI-inspired e-commerce platform specialized in electronic devices. The document is intended for the development team, project stakeholders, instructors, and future maintainers. It serves as the primary reference for system modeling, design, implementation of the prototype, and validation activities required by CSE327.

1.2 || Why We Built It This Way

Projukti Mart is a multi-sided online marketplace that enables customers to discover, evaluate, and purchase electronic products (Laptops, Desktop PCs, Keyboards, Headphones, Cables, and Stands), allows sellers to manage product listings and fulfill orders, and provides platform administrators with oversight and policy-control capabilities. The system embeds a mandatory **AI-Assisted Semantic Product Search** capability that accepts natural-language queries and returns ranked, relevant products. On product listing and detail pages, the system also surfaces light, personalized recommendations based on the user's recent browsing and purchase history. When the AI component is unavailable, the platform degrades gracefully to conventional keyword/filter search.

In scope:

- User registration, authentication, and profile management.
- Product catalog browsing, filtering, and AI-assisted search.
- Shopping cart, checkout, order placement, and order history.
- Product ratings and reviews.
- Seller product listing management and order processing.
- Basic sales summaries for sellers.
- Administrator management of categories, sellers, users, and flagged content.
- AI semantic search + personalized recommendations with fallback behavior.

Out of scope (for this project):

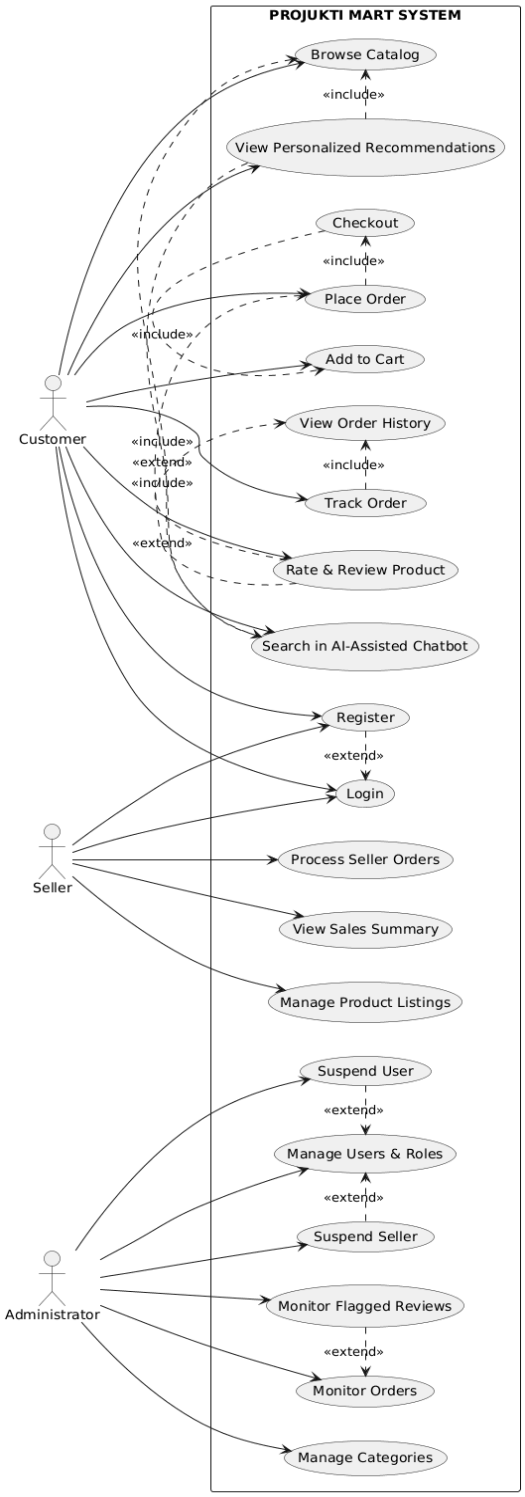
• Payment gateway integration with real financial institutions (mock payment is acceptable).
• Physical logistics/shipping carrier integration.
• Mobile native applications (responsive web is sufficient).
• Multi-language support beyond English.

1.3 || Scope of This Report

This report covers the system architecture, core domain models, role-based workflows, REST API interfaces, database designs, testing approaches, UI wireframes, and known limitations for Projukti Mart.

2. Modeling & Design Diagrams

2.1 || Use Case Diagram



2.2 || Use Case Descriptions

UC-01: AI-Assisted Product Search

Field	Description
Actor	Customer (primary), Guest
Precondition	User is on any page that contains the search bar. The catalog contains products.
Trigger	User types a natural-language query (e.g., “lightweight laptop for programming under 80k”) and submits.
Main Flow	1. The system sends a query + optional context to the AI Semantic Search service. 2. AI returns a ranked product list with scores. 3. If confidence is acceptable, the system displays results labeled “AI Search results.” 4. Users may click a product to view details.
Alternate / Exception	• AI timeout or low confidence → system automatically runs keyword search and shows “Showing keyword results (AI unavailable)." • Empty result set → display “No products matched your query” with suggestions to broaden filters.
Postcondition	Search results are displayed. The query is optionally logged for future personalization.

UC-02: Add to Cart & Checkout

Field	Description
Actor	Customer (must be logged in for checkout)
Precondition	The product exists and has stockQty > 0. Users may already have items in their cart.
Trigger	The user clicks “Add to Cart” on a product page or listing, then proceeds to Checkout.
Main Flow	1. The system validates the requested quantity ≤ available stock. 2. Items are added/updated in the cart. 3. The user navigates to the cart, reviews items, and updates quantities if desired. 4. The user proceeds to checkout, confirms the shipping address, and orders a

	summary. 5. The system re-validates stock, creates Order + OrderItems, decrements stock, and clears the cart. 6. The confirmation page is shown with the order ID.
Alternate / Exception	• Quantity exceeds stock → error message, quantity capped or rejected. • Stock changes between cart and checkout → system re-checks and notifies users of any items that become unavailable.
Postcondition	The order is recorded in the Pending/Confirmed state. Stock is reduced. The cart is empty.

UC-03: Seller Manages Product Listing

Field	Description
Actor	Seller
Precondition	The seller is authenticated, and the account is active.
Trigger	The seller navigates to "My Products" and chooses Create / Edit / Deactivate
Main Flow	1. Seller fills out product form (title, description, brand, category, price, stock, key specs, images). 2. The system validates required fields and price > 0, stock ≥ 0. 3. The product is saved and becomes visible in the catalog (if active). 4. Seller may later edit any field or set the status to Inactive.
Alternate / Exception	• Validation failure → form redisplays with error messages. • Attempt to edit another seller's product → authorization error.
Postcondition	Product records are created or updated. The catalog reflects the change.

UC-04: Verify and Process an Order (Seller)

Field	Description
Actor	Seller
Precondition	An order exists that contains at least one product belonging to the seller.
Trigger	Seller opens the "Incoming Orders" dashboard and selects an order.
Main Flow	1. The system displays order details filtered to the seller's line items. 2. Seller updates status (e.g., Confirmed → Shipped) and may add tracking notes. 3. The system records the status change and timestamp. 4. Customers can later see the updated status in the order history.
Alternate / Exception	• Seller attempts to change an order that contains none of their products → access denied.
Postcondition	Order status is updated. Audit trail is retained.

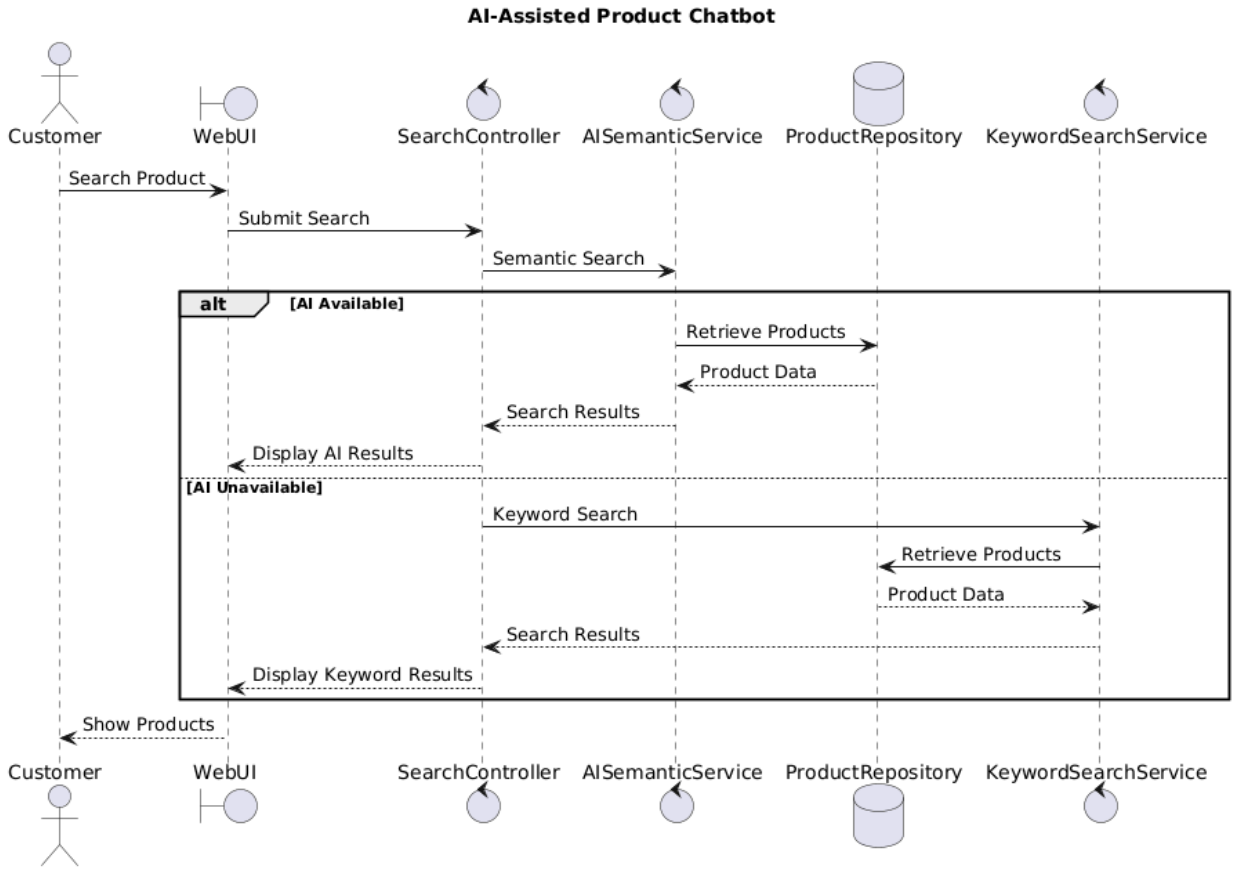
UC-05: View Personalized Recommendations

Field	Description
Actor	Customer (logged-in or anonymous session)
Precondition	The user has browsing or purchase history, or is viewing a product detail page.
Trigger	The user lands on the Home page or Product Detail page.
Main Flow	1. System requests recommendations from the Recommendation service (userId / session + optional context product). 2. Service returns a list of product IDs. 3. System renders a "Recommended for you" or "Customers also viewed" carousel. 4. Users may click any recommended product.
Alternate / Exception	• Service returns an empty list or is unavailable → carousel is hidden; no error is shown to the user.
Postcondition	Recommendations are displayed (or silently omitted).

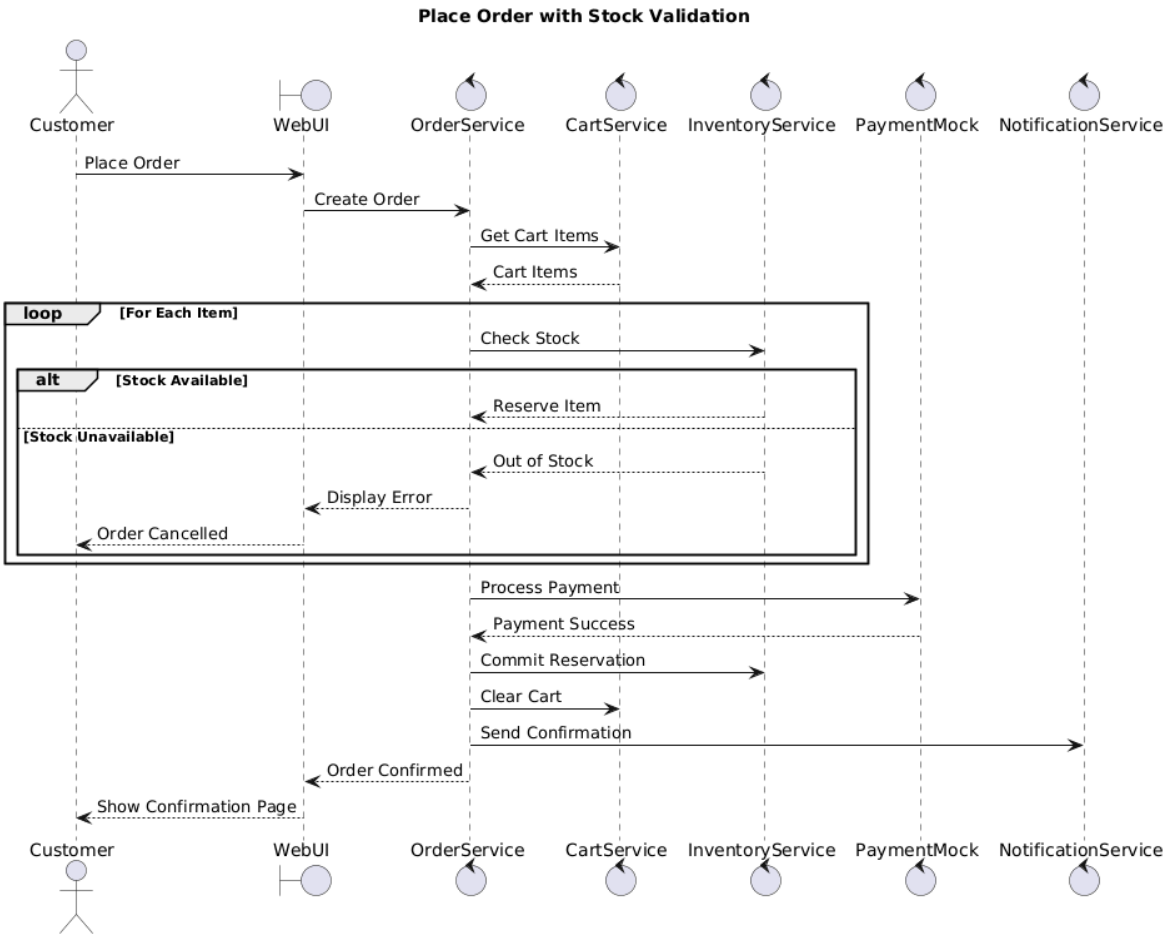
2.3 || Sequence Diagrams

Two major interactions are documented. The first concerns the AI component; the second covers the critical order-placement path, including stock validation.

Sequence Diagram 1: AI-Assisted Product Search

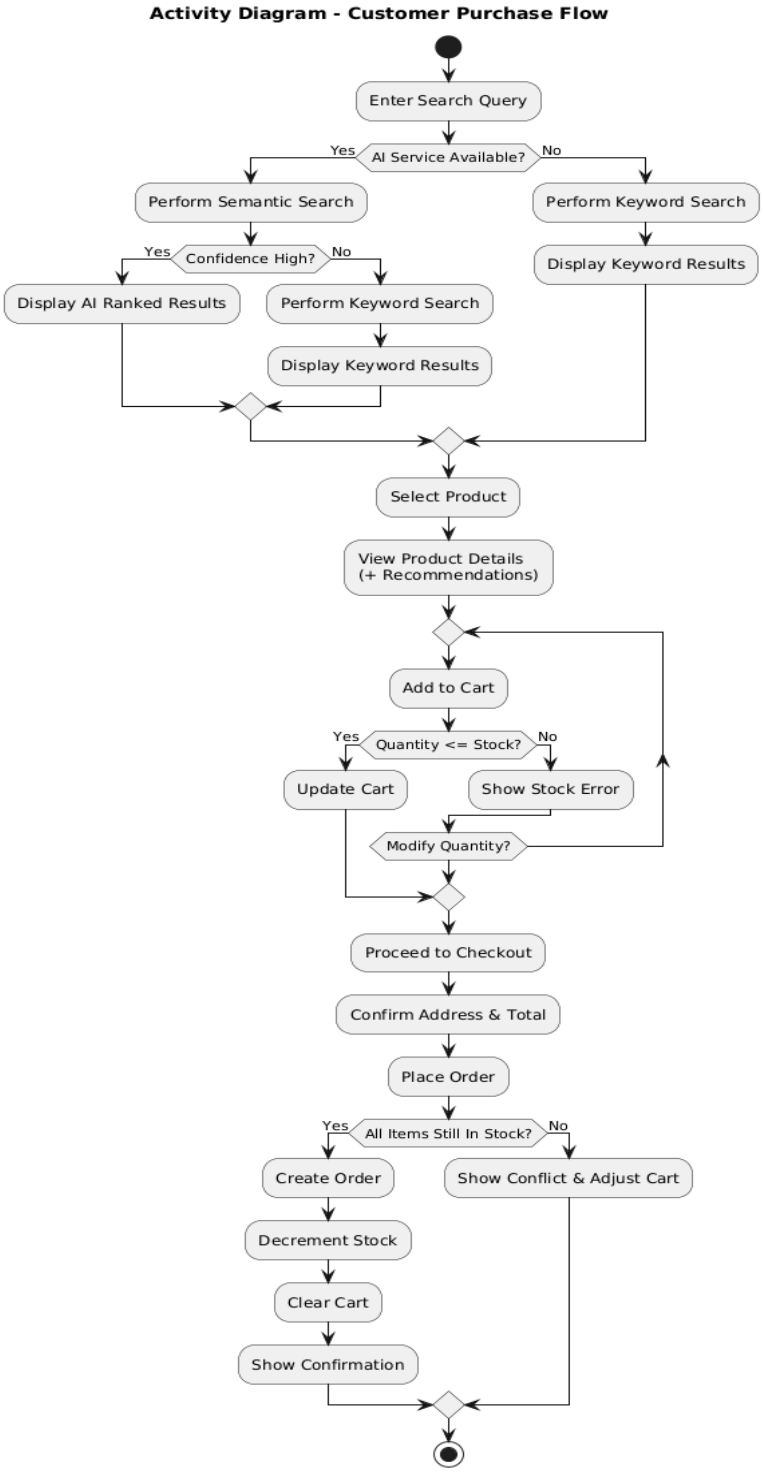


Sequence Diagram 2: Place Order (with stock validation)

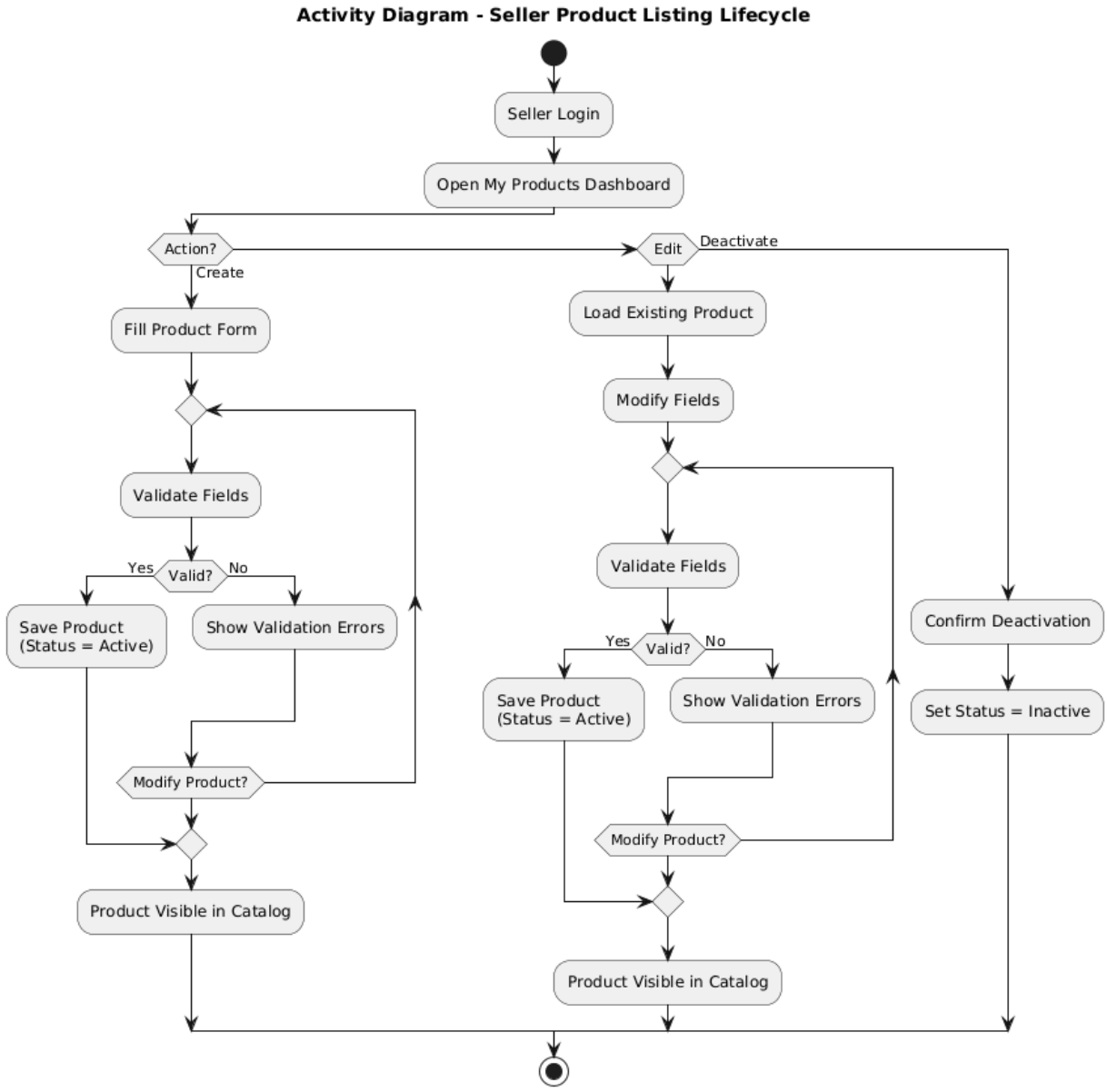


2.4 || Activity Diagrams

Activity Diagram 1: Customer Purchase Flow (Search → Order)

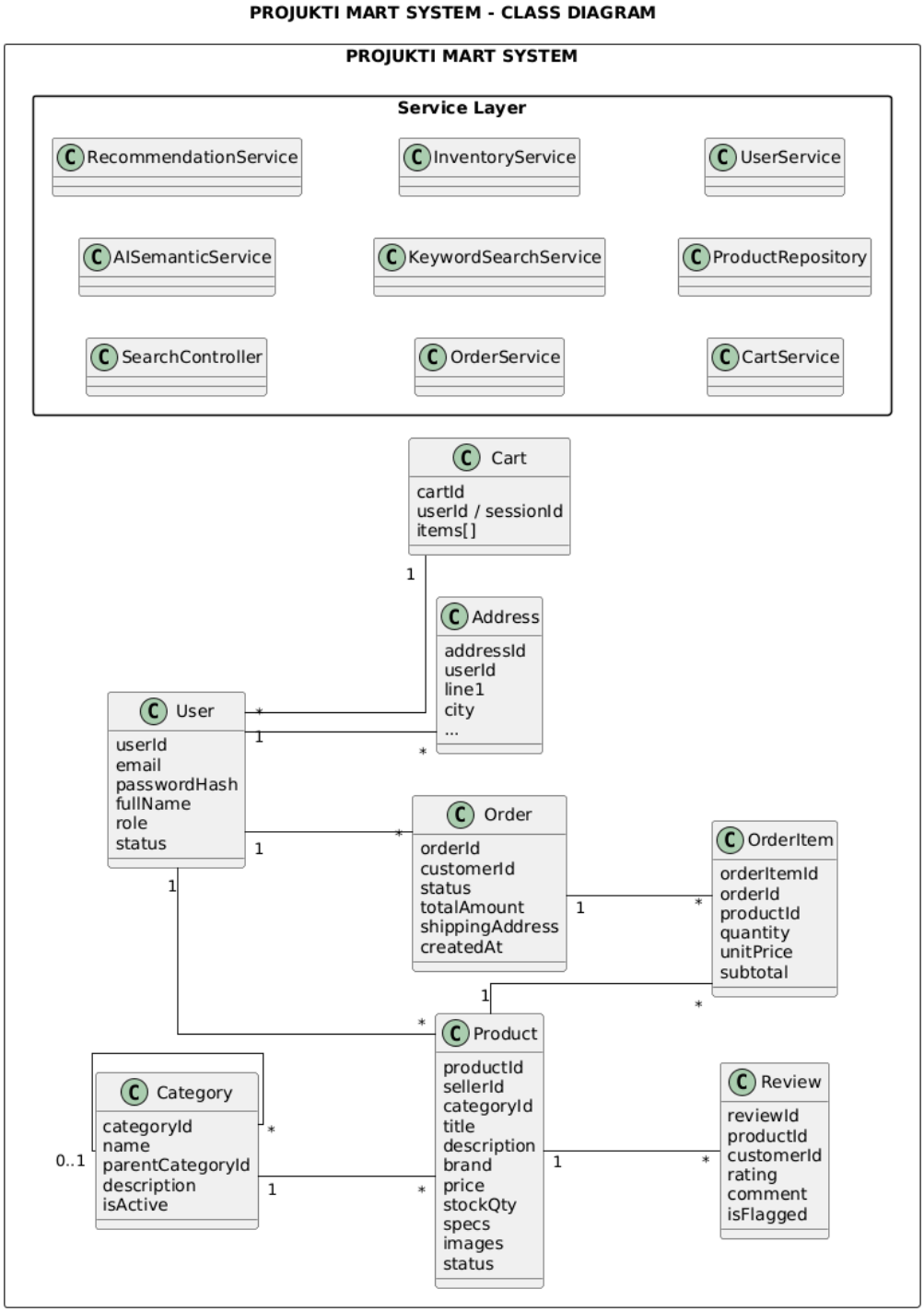


Activity Diagram 2: Seller Product Listing Lifecycle



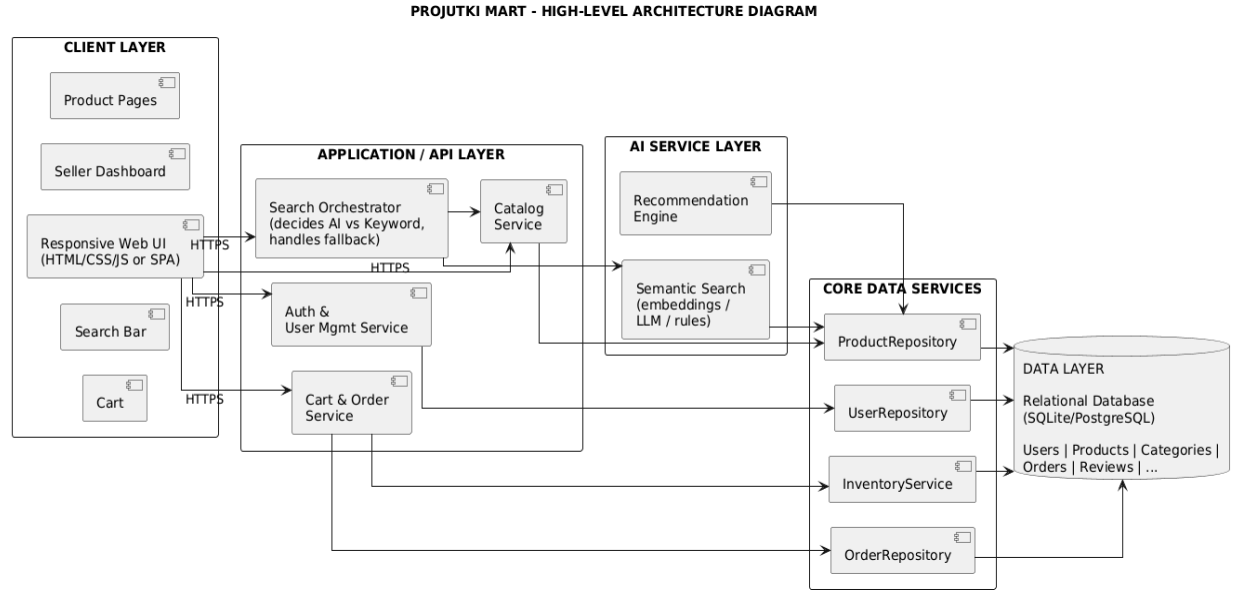
2.5 || Class Diagram

The principal domain and service classes are listed below, along with key attributes and relationships. In a formal UML tool, this would be drawn with association, aggregation, and inheritance connectors.



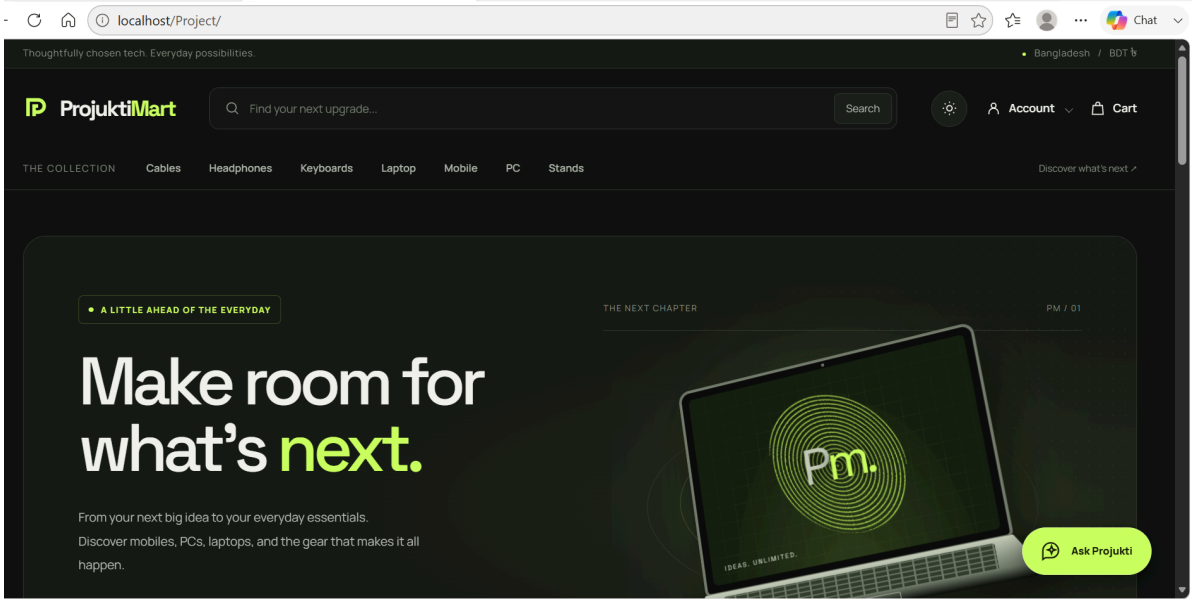
2.6 || High-Level Architecture Diagram

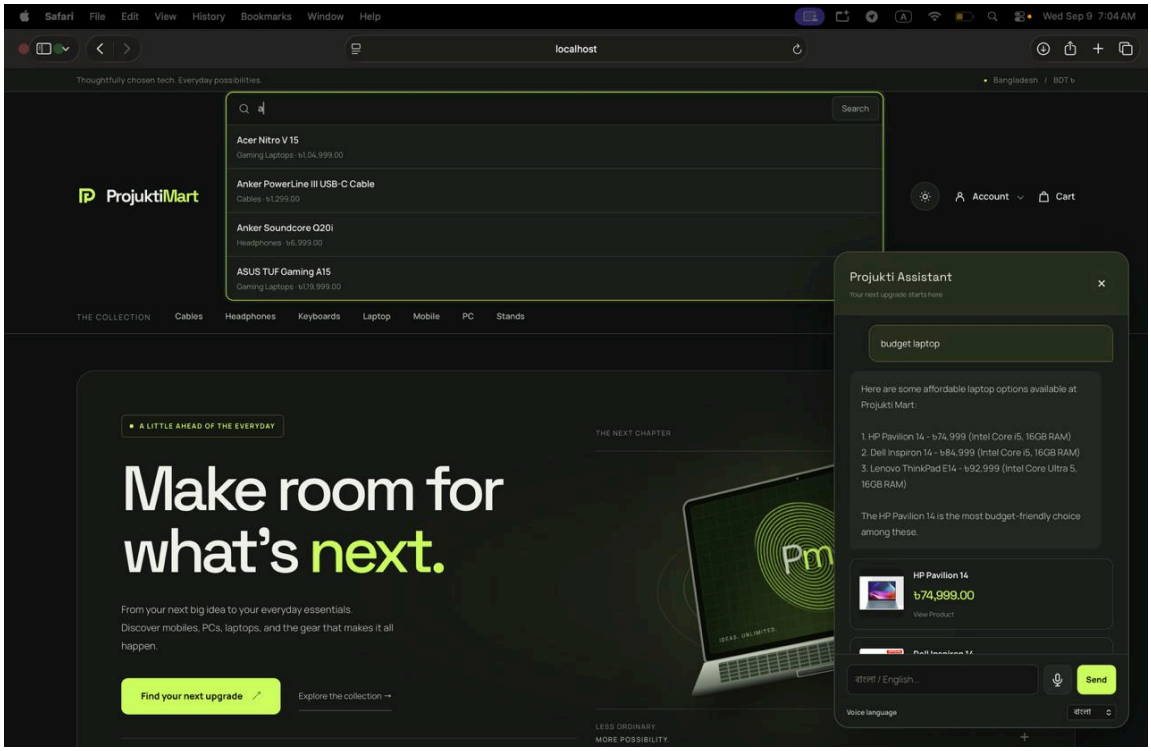
The architecture follows a layered approach, with a clearly isolated AI component to enable graceful degradation.



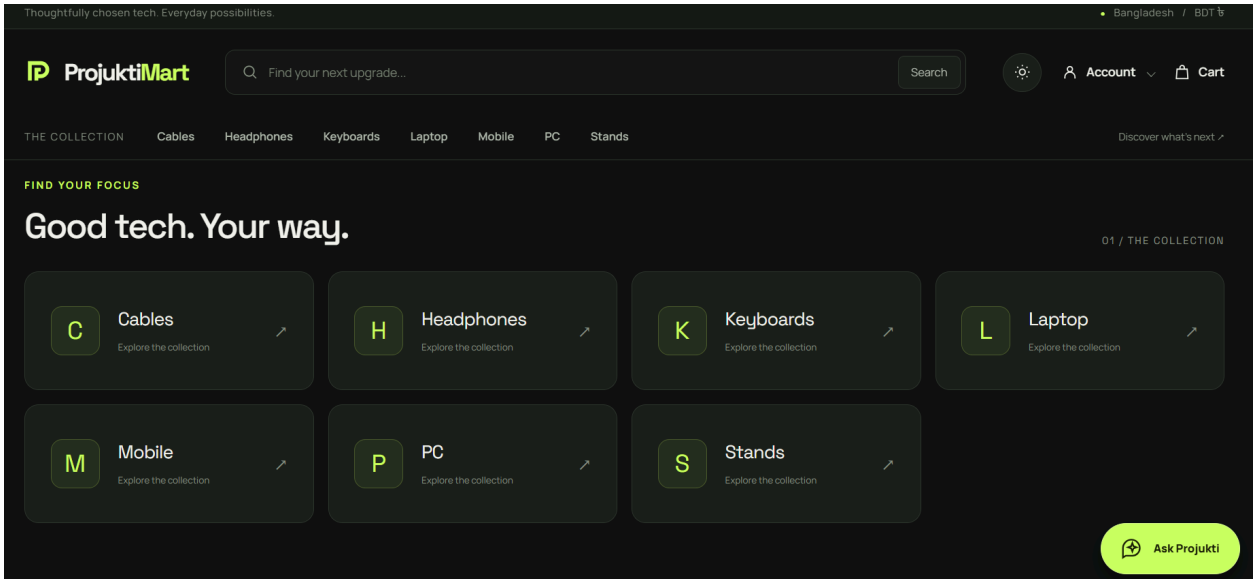
2.7 || UI Wireframes / Mockups

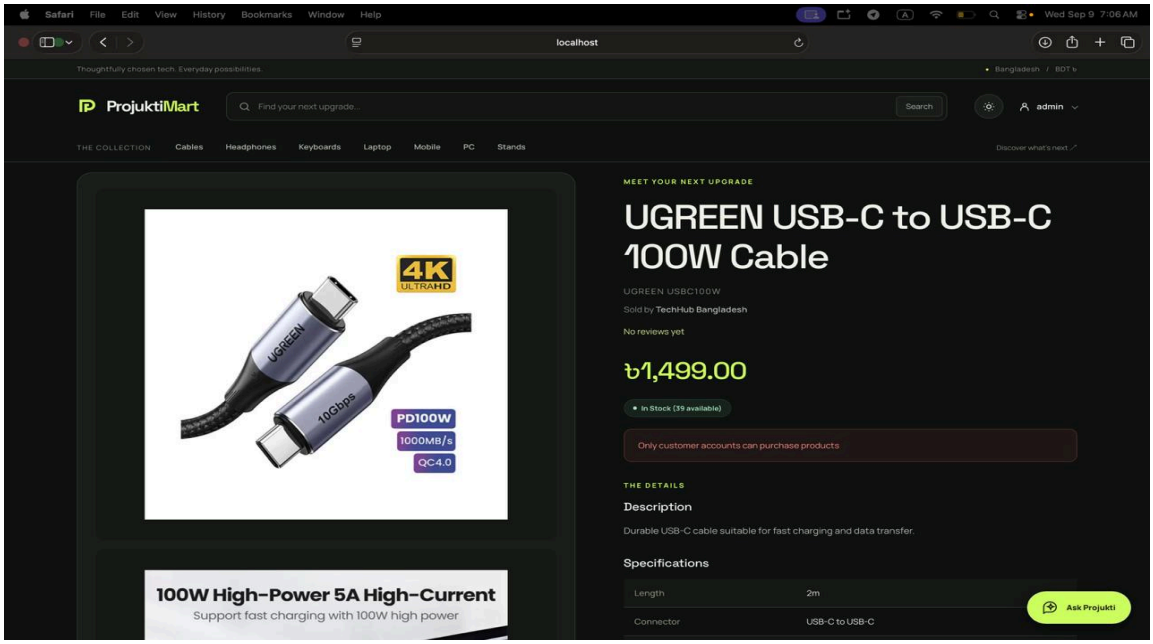
Screen 1: Product Search / AI Assistant Interface



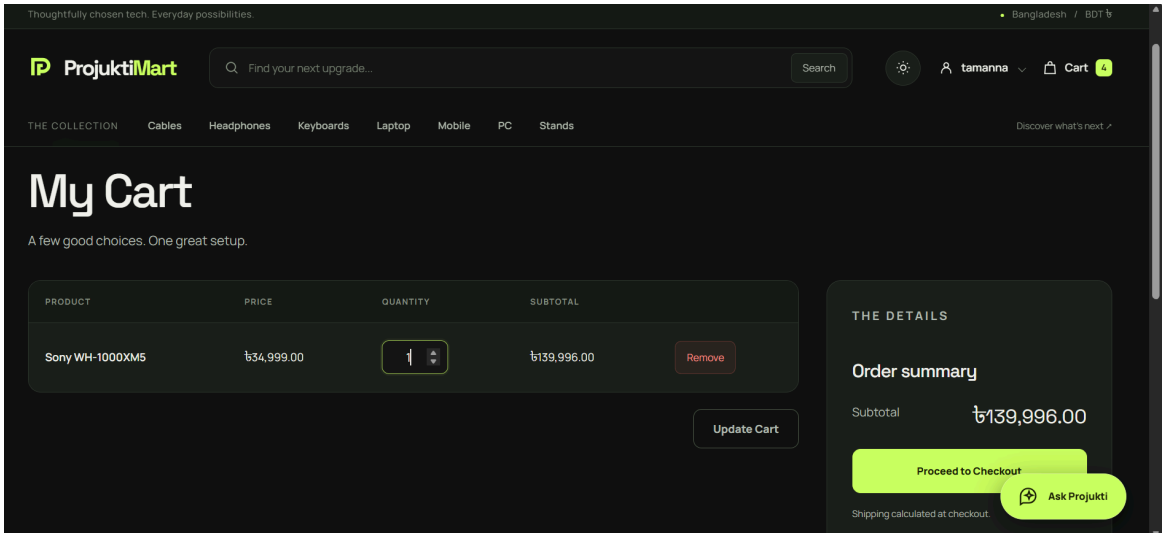


Screen 2: Product Listing / Detail Page with Recommendations

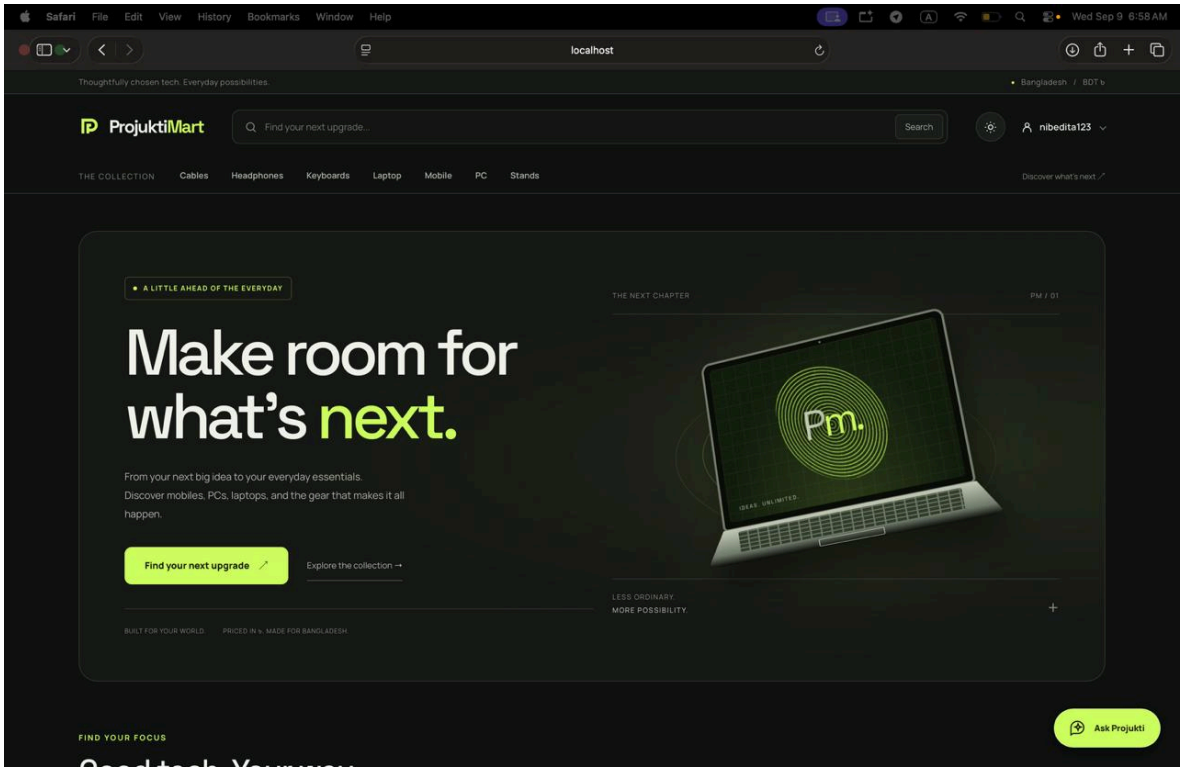




Screen 3: Cart & Checkout Screen



Screen 4: Seller Product Management Dashboard



3. Module Breakdown

3.1 || Authentication & Profile Management

Authentication utilizes email-based login combined with single-way password hashing. System sessions enforce Role-Based Access Control across three core roles: Customer, Seller, and Administrator.

3.2 || AI Semantic Search & Fallback Engine

The search subsystem accepts free-text natural-language input. If the AI service responds within target performance metrics ($< 5\text{ seconds}$) with high confidence, ranked results are displayed. Otherwise, the system automatically redirects the query to SQL/keyword operations.

3.3 || Shopping Cart & Inventory Processing

The cart subsystem maintains selected items per user or guest session. Atomic checks ensure that order quantities never exceed active stockQty during both cart addition and final transaction execution.

3.4 || Seller Operations & Fulfillment

Sellers access dedicated endpoints to create, edit, or deactivate electronics listings. The order fulfillment screen allows sellers to update order line-item statuses (e.g., Confirmed, Shipped) for items belonging strictly to their store.

3.5 || Administration & Platform Moderation

Administrators hold platform-wide oversight, enabling them to create/deactivate catalog categories, modify user account statuses, manage seller approvals, and moderate flagged reviews.

4. Role & Permission Model

Action	Customer	Seller	Admin
Register / Login	Yes	Yes	Yes

Browse & Search catalog	Yes	Yes	Yes
Use AI semantic search	Yes	Yes	Yes
Add to cart / Place order	Yes	No*	No*
View own orders	Yes	Yes (as seller)	Yes (all)
Write review	Yes (purchased)	No	No
Manage own products	No	Yes	Yes (override)
Process own orders	No	Yes	Yes
Manage categories	No	No	Yes
Manage users/roles	No	No	Yes
Flag / moderate reviews	No	No	Yes

* Sellers and Admins may also act as customers if they hold a dual role; otherwise, purchase is restricted to the Customer role.

5. Rest API Interface

Method	Endpoint	Access Role	Description
Post	/api/auth/register	Public	Registers a new customer or seller account.
Post	/api/auth/login	Public	Authenticates credentials and returns a session token.
Get	/api/products	All	Retrieves product listings with category and price filters.
Post	api/search/semantic	All	Processes natural language

			query via AI or fallback search.
Get	/api/recommendations	All	Fetches personalized product recommendation ids.
Post	/api/cart/add	Customer	Adds an item to cart after verifying availability.
Post	api/orders/checkout	Customer	Executes orders placement and decrements product stock.
Post	api/seller/products	Seller	Creates or updates product listings and stock levels.
Patch	api/admin/users/:id	Admin	Updates user status.

6. Setup & Running the application

6.1 || Requirements

1. **Node.js:** v18.0 or higher
2. **Database Engine:** MySQL 8.0+ or PostgreSQL
3. **Package Manager:** npm / yarn

6.2 || Steps

1. Clone the project repository locally.
2. Install system dependencies:

   npm install

3. Configure environment variables in .env

   PORT=3000

DB_HOST=localhost

DB_USER=root

DB_PASS=password

DB_NAME=projuktimart_db

AI_SERVICE_URL=http://localhost:5000

4. Seed the database with product samples and default roles:

   node scripts/seed_db.js

5. Start the backend application server:

   npm start

6. Access http://localhost:3000 in your web browser.

7. Limitations and Future Enhancements

7.1 || Limitations

- Payment Processing: Integrates mock payment verification rather than active banking gateways.
- Real-time Push Notifications: Order status changes rely on manual client polling or page refreshes rather than WebSockets.
- Language Support: Prototype restricted to English query processing.

7.2 || Future Enhancements

- Transition frontend components to modern rendering frameworks (e.g., React or Vue).
- Add WebSocket integrations for real-time seller order updates.
- Expand multi-language NLP support to process localized Bengali queries.

8. Frontend Design

8.1 Visual Direction

Projukti Mart uses a modern technology-focused e-commerce design with a clean and consistent layout. The interface uses reusable components such as a common header, navigation bar, search bar, product cards, category tiles, and footer. Product information is presented clearly through images, names, brands, prices, and detail buttons. CSS is used to maintain consistent typography, spacing, buttons, and card layouts across the website.

8.2 Responsiveness

The website uses responsive layouts and flexible product grids so that the content can adapt to different screen sizes. The product cards, category sections, navigation, and other main components adjust according to the available screen width. The current design focuses mainly on desktop and laptop screens, while further optimization for smaller mobile screens can be added in future iterations.

8.3 Product Search and Navigation

Projukti Mart includes a search bar with dynamic product suggestions. As users type, matching products are retrieved and displayed, allowing them to quickly access product details. Products are also organized into categories, which are loaded dynamically from the database and displayed in the navigation, home page, and footer.

8.4 Role-Based Interface and Chatbot

The frontend provides different navigation options based on the user's role. Customers can access shopping and order features, sellers can manage products and sales, and administrators can access the admin dashboard. A floating **Projukti Assistant** chatbot is also included, allowing users to ask questions about products, categories, checkout, sellers, and other parts of the system.

9. Testing

9.1 Test Credentials

The project includes test accounts for the main user roles so that the different parts of the system can be tested easily. Separate accounts can be used for customers, sellers, and administrators to verify their respective features and access permissions. These accounts allow us to test login, product management, shopping cart, orders, and administrative functions without creating new accounts every time.

9.2 Product and Order Testing

We tested the product catalog by checking that products are correctly displayed with their names, brands, prices, images, and categories. The search and category navigation were also tested to verify that users can find the required products. The cart and checkout flow were tested by adding products, reviewing cart items, and placing orders.

9.3 Role Gating

Role-based access was tested by logging in with customer, seller, and administrator accounts. We verified that each role receives the appropriate navigation options and access to its permitted features. For example, customers can browse products and manage orders, sellers can manage their own products and view sales information, while administrators can access the admin dashboard and administrative functions. This helped verify that restricted functionality is not unnecessarily exposed to other users.

10. Setup and Running the Application

10.1 What You Need

The following software and tools are required to run the Projukti Mart application:

- PHP 8.2 or later
- MySQL/MariaDB database server
- XAMPP, which can be used to provide Apache and MySQL/MariaDB
- phpMyAdmin for importing and managing the database
- A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox

The application uses PHP for server-side development and MySQL/MariaDB for database management. The database connection is configured in the db.php file, which connects to a database named "projukti_mart".

10.2 Getting Started

Follow the steps below to set up and run the application:

1. Extract the Projukti Mart project folder into the XAMPP htdocs directory.
2. Open the XAMPP Control Panel and start the Apache and MySQL services.
3. Open phpMyAdmin by visiting:

http://localhost/phpmyadmin

4. Create a new database named:

projukti_mart

5. Import the projukti_mart.sql file into the newly created database. This SQL file contains the required database tables, relationships, indexes, and sample data for the application.
6. Make sure that all project files and folders remain in their original directory structure.
7. Check the database configuration in the db.php file. The default configuration uses the following settings:

Host: localhost
Username: root
Password: empty
Database: projukti_mart

8. Open a web browser and navigate to the project directory. For example, if the project folder is named "projukti_mart", visit:

http://localhost/projukti_mart/

9. The homepage allows users to browse available products and product categories.
10. Customers can register or log in to the system, browse products, add products to their shopping cart, proceed to checkout, and manage their orders.
11. Sellers can manage their products, stock, and orders through the seller section of the application.
12. Administrators can manage users, products, categories, orders, and other administrative functions.
13. Log in using one of the test accounts provided with the project to test the different user roles and functionalities.

10.3 Main Application Workflow

The main customer workflow of Projukti Mart is:

Register/Login → Browse Products → View Product → Add to Cart → Checkout → Place Order → Track Order

Customers can browse products by category or search for specific products. After selecting a product, they can add it to their shopping cart and adjust the desired quantity. During checkout, the system verifies the customer's information, shipping address, and product availability before creating an order.

Once an order is successfully placed, the order information and its individual items are stored in the database. The customer's cart is then cleared, and the order can be tracked through the order management section.

10.4 Stopping the Application

To stop the application, close the browser after finishing the session. The Apache and MySQL services can then be stopped from the XAMPP Control Panel if they are no longer required.

11. Limitations and What We'd Do Next

11.1 What Needs Fixing Before Production

• The project is currently designed as a CSE327 academic prototype and would require additional security testing before real deployment.

• Password and authentication handling would need stronger production-level security measures, including secure password hashing and improved session management.

• The payment process currently uses a mock payment system rather than a real payment gateway.

• The chatbot provides project-related assistance but is limited by the available project knowledge and endpoint implementation.

• The current responsive design could be further optimized for very small mobile screens.

11.2 What We'd Add Given More Time

• Integrate a real payment gateway such as SSLCommerz or another supported payment provider.

• Improve the chatbot with a more advanced knowledge base and natural-language responses.

• Add product reviews, ratings, wishlist functionality, and product comparison.

• Improve mobile responsiveness and develop a dedicated mobile-friendly interface.

• Add stronger security features such as improved authentication, input validation, and protection against common web attacks.

• Add more detailed sales analytics and reporting for sellers and administrators.

12. Conclusion

Projukti Mart is a PHP and MySQL/MariaDB-based e-commerce application designed for selling technology products online. It provides separate functionalities for customers, sellers, and administrators, including product browsing, shopping cart management, checkout, order processing, product management, and user management.

The project demonstrates the practical use of database management, authentication, role-based access control, and e-commerce functionality. Although it is developed as a university project, it provides a solid foundation that could be improved in the future with online payment integration, stronger security, real-time order tracking, and advanced product recommendations